# Functions and an Introduction to Recursion

Chapter 6 of C++ How to Program, 9/e

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

## OBJECTIVES

In this chapter you'll:

- Construct programs modularly from functions.
- Use common math library functions.
- Use function prototypes to declare a function.
- Use random-number generation to implement game-playing applications.
- Use C++14 digit separators to make numeric literals more readable
- Understand how the visibility of identifiers is limited to specific regions of programs.
- Understand how the function call/return mechanism is supported by the function-call stack and activation records.
- Understand the mechanisms for passing data to functions and returning results.
- Use inline functions, references and default arguments.
- Define with the same name overloaded functions that perform different tasks based on the number and types of their arguments.
- Define function templates that can generate families of overloaded functions.
- Write and use recursive functions.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

**6.1** Introduction

**6.2** Program Components in C++

**6.3** Math Library Functions

**6.4** Function Prototypes

**6.5** Function-Prototype and Argument-Coercion Notes

- 6.5.1 Function Signatures and Function Prototypes
- 6.5.2 Argument Coercion
- 6.5.3 Argument-Promotion Rules and Implicit Conversions

**6.6** C++ Standard Library Headers

**6.7** Case Study: Random-Number Generation

- 6.7.1 Rolling a Six-Sided Die
- 6.7.2 Rolling a Six-Sided Die 60,000,000 Times
- 6.7.3 Randomizing the Random-Number Generator with `srand`
- 6.7.4 Seeding the Random-Number Generator with the Current Time
- 6.7.5 Scaling and Shifting Random Numbers

**6.8** Case Study: Game of Chance; Introducing Scoped **enum**s


©1992–2017 by Pearson Education, Inc. All Rights Reserved.

6.9 C++11 Random Numbers
6.10 Scope Rules
6.11 Function-Call Stack and Activation Records
6.12 Inline Functions
6.13 References and Reference Parameters
6.14 Default Arguments
6.15 Unary Scope Resolution Operator
6.16 Function Overloading
6.17 Function Templates
6.18 Recursion
6.19 Example Using Recursion: Fibonacci Series
6.20 Recursion vs. Iteration
6.21 Wrap-Up

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.1 Introduction

- Develop and maintain a large program by constructing it from small, simple pieces, or components.
  - divide and conquer.
- Emphasize how to declare and use functions to facilitate the design, implementation, operation and maintenance of large programs.
- Function prototypes and how the compiler uses them to convert the type of an argument in a function call to the type specified in a function's parameter list, if necessary.
- Simulation techniques with random number generation.
- C++'s scope rules.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.1 Introduction (cont.)

- How C++ keeps track of which function is currently executing, how parameters and other local variables of functions are maintained in memory and how a function knows where to return after it completes execution.
- Function overloading.
- Function templates.
- Recursion.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.2 Program Components in C++

- C++ programs are typically written by combining "prepackaged" functions and classes available in the C++ Standard Library with new functions and classes you write.
- The C++ Standard Library provides a rich collection of functions.
- Functions allow you to modularize a program by separating its tasks into self-contained units.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Software Engineering Observation 6.1

*To promote software reusability, every function should be limited to performing a single, well-defined task, and the name of the function should express that task effectively.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.2 Program Components in C++ (cont.)

- A function is invoked by a function call, and when the called function completes its task, it either returns a *result* or simply returns *control* to the caller.
- An analogy is the hierarchical form of management (Figure 6.1).
  - A boss (similar to the calling function) asks a worker (similar to the called function) to perform a task and report back (i.e., return) the results after completing the task.
  - The boss does not know how the worker performs its tasks.
  - The worker may also call other workers, unbeknownst to the boss.
- This *hiding of implementation details* promotes good software engineering.
- The relationship does not need to be hierarchical, but often it is, which makes it easier to test, debug, update and maintain programs.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

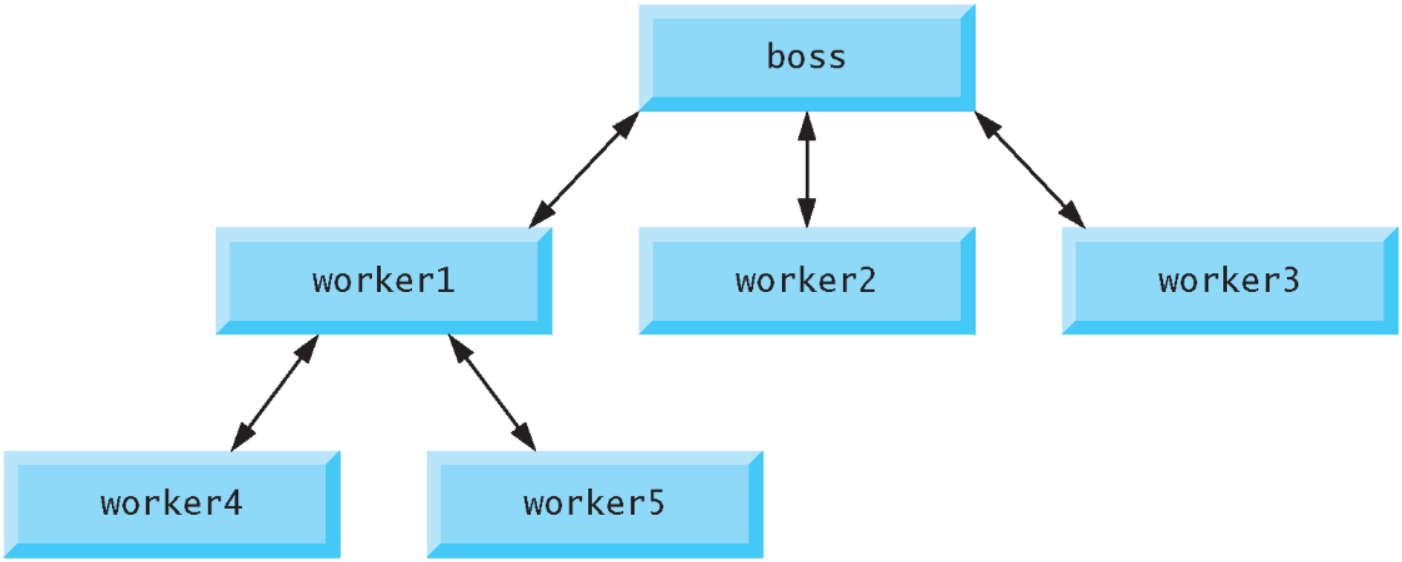



**Fig. 6.1** | Hierarchical boss-function/worker-function relationship.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.3 Math Library Functions

- Sometimes functions are *not* members of a class.
  - Called global functions.
- The `<cmath>` header provides a collection of functions that enable you to perform common mathematical calculations.
- All functions in the `<cmath>` header are global functions—therefore, each is called simply by specifying the name of the function followed by parentheses containing the function's arguments.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Error-Prevention Tip 6.1

*Do not call* `sqrt` *with a negative argument. For industrial-strength code, always check that the arguments you pass to math functions are valid.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.3 Math Library Functions (cont.)

- Some math library functions are summarized in Fig. 6.2.
  - In the figure, the variables `x` and `y` are of type `double`.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

| Function | Description | Example |
|---|---|---|
| `ceil(x)` | rounds $x$ to the smallest integer not less than $x$ | `ceil(9.2)` is `10.0`<br>`ceil(-9.8)` is `-9.0` |
| `cos(x)` | trigonometric cosine of $x$ ($x$ in radians) | `cos(0.0)` is `1.0` |
| `exp(x)` | exponential function $e^x$ | `exp(1.0)` is `2.718282`<br>`exp(2.0)` is `7.389056` |
| `fabs(x)` | absolute value of $x$ | `fabs(5.1)` is `5.1`<br>`fabs(0.0)` is `0.0`<br>`fabs(-8.76)` is `8.76` |
| `floor(x)` | rounds $x$ to the largest integer not greater than $x$ | `floor(9.2)` is `9.0`<br>`floor(-9.8)` is `-10.0` |
| `fmod(x, y)` | remainder of $x/y$ as a floating-point number | `fmod(2.6, 1.2)` is `0.2` |
| `log(x)` | natural logarithm of $x$ (base $e$) | `log(2.718282)` is `1.0`<br>`log(7.389056)` is `2.0` |
| `log10(x)` | logarithm of $x$ (base 10) | `log10(10.0)` is `1.0`<br>`log10(100.0)` is `2.0` |
| `pow(x, y)` | $x$ raised to power $y$ ($x^y$) | `pow(2, 7)` is `128`<br>`pow(9, .5)` is `3` |

**Fig. 6.2** | Math library functions. (Part 1 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

| Function | Description | Example |
|---|---|---|
| `sin(x)` | trigonometric sine of $x$ ($x$ in radians) | `sin(0.0)` is `0` |
| `sqrt(x)` | square root of $x$ (where $x$ is a nonnegative value) | `sqrt(9.0)` is `3.0` |
| `tan(x)` | trigonometric tangent of $x$ ($x$ in radians) | `tan(0.0)` is `0` |

**Fig. 6.2** | Math library functions. (Part 2 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.4 Function Pototypes

- In this section, we create a user-defined function called `maximum` (Fig. 6.3) that returns the largest of its three `int` arguments.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.3: fig06_03.cpp
2  // maximum function with a function prototype.
3  #include <iostream>
4  #include <iomanip>
5  using namespace std;
6
7  int maximum(int x, int y, int z); // function prototype
8
9  int main() {
10    cout << "Enter three integer values: ";
11    int int1, int2, int3;
12    cin >> int1 >> int2 >> int3;
13
14    // invoke maximum
15    cout << "The maximum integer value is: "
16       << maximum(int1, int2, int3) << endl;
17 }
18
```

**Fig. 6.3** | `maximum` function with a function prototype. (Part 1 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
19  // returns the largest of three integers
20  int maximum(int x, int y, int z) {
21     int maximumValue{x}; // assume x is the largest to start
22
23     // determine whether y is greater than maximumValue
24     if (y > maximumValue) {
25        maximumValue = y; // make y the new maximumValue
26     }
27
28     // determine whether z is greater than maximumValue
29     if (z > maximumValue) {
30        maximumValue = z; // make z the new maximumValue
31     }
32
33     return maximumValue;
34  }
```

**Fig. 6.3** | `maximum` function with a function prototype. (Part 2 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
Enter three integer grades: 86 67 75
The maximum integer value is: 86
```

```
Enter three integer grades: 67 86 75
The maximum integer value is: 86
```

```
Enter three integer grades: 67 75 86
The maximum integer value is: 86
```

**Fig. 6.3** | `maximum` function with a function prototype. (Part 3 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.4 Function Prototypes (cont.)

- In preceding chapters, we created classes with various member functions.
- We defined each class in a header (.h) and included it before `main` in a program's source-code file.
- Doing this ensures that the class (and thus its member functions) is defined before `main` creates and manipulates objects of that class.
- The compiler then can ensure that we call each class's constructors and member functions correcly—for example, passing each the correct number and types of arguments.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.4 Function Prototypes (cont.)

- For a function not defined in a class, you must either define the function before using it or you must declare that the function exists, as we do in line 7 of Fig. 6.3:
  - `int maximum(int x, int y, int z); // function prototype`
- This is a function prototype, which describes the `maximum` function without revealing its implementation.
- A function prototype is a declaration of a function
  - tells the compiler the function's name, its return type and the types of its parameters.
- This function prototype indicates that the function returns an `int`, has the name `maximum` and requires three `int` parameters to perform its task.
- The function prototype is the same as the first line of the corresponding function definition (line 20), but ends with a required semicolon.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Good Programming Practice 6.1

*Parameter names in function prototypes are optional (they're ignored by the compiler), but many programmers use these names for documentation purposes. We used parameter names in Fig. 6.3's function prototype for demonstration purposes, but generally we do not use them in this book's subsequent examples.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.4 Function Prototypes (cont.)

- When compiling the program, the compiler uses the prototype to
  - Ensure that `maximum`'s first line matches its prototype.
  - Check that the call to `maximum` contains the correct number and types of arguments, and that the types of the arguments are in the correct order.
  - Ensure that the value returned by the function can be used correctly in the expression that called the function.
  - Ensure that each argument is consistent with the type of the corresponding parameter.
    - If the arguments passed to a function do not match the types specified in the function's prototype, the compiler attempts to convert the arguments to those types.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Common Programming Error 6.1

*Declaring function parameters of the same type as* `int x, y` *instead of* `int x, int y` *is a syntax error—a type is required for each parameter in the parameter list.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Common Programming Error 6.2

*Compilation errors occur if the function prototype, header and calls do not all agree in the number, type and order of arguments and parameters, and in the return type.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Software Engineering Observation 6.2

*A function that has many parameters may be performing too many tasks. Consider dividing the function into smaller functions that perform the separate tasks. Limit the function header to one line if possible.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.4 Function Prototypes (cont.)

- Multiple parameters are specified in both the function prototype and the function header as a comma-separated list, as are multiple arguments in a function call.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Portability Tip 6.1

*The commas used in line 16 of Fig. 6.3 to separate the arguments to function* `maximum` *are* not *comma operators as discussed in Section 5.5. The comma operator guarantees that its operands are evaluated left to right. The order of evaluation of a function's arguments, however, is* not *specified by the C++ standard. Thus, different compilers can evaluate function arguments in different orders.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Portability Tip 6.2

*Sometimes when a function's arguments are expressions, such as those with calls to other functions, the order in which the compiler evaluates the arguments could affect the values of one or more of the arguments. If the evaluation order changes between compilers, the argument values passed to the function could vary, causing subtle logic errors.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Error-Prevention Tip 6.2

*If you have doubts about the order of evaluation of a function's arguments and whether the order would affect the values passed to the function, evaluate the arguments in separate assignment statements before the function call, assign the result of each expression to a local variable, then pass those variables as arguments to the function.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.4 Function Definitions with Multiple Parameters (cont.)

- In a function that does not return a result (i.e., it has a `void` return type), we showed that control returns when the program reaches the function-ending right brace.
- You also can explicitly return control to the caller by executing the statement
  - `return;`

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.5 Function-Prototypes and Argument-Coercion Notes

- A function prototype is required unless the function is defined before it's used.
- When you use a standard library function like sqrt, you do not have access to the function's definition, therefore it cannot be defined in your code before you call the function.
- Instead, you must include its corresponding header (`<cmath>`), which contains the function's prototype.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Common Programming Error 6.3

*If a function is defined* before *it's called, then its definition also serves as the function's prototype, so a separate prototype is unnecessary. If a function is called* before *it's defined, and that function does not have a function prototype, a compilation error occurs.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Software Engineering Observation 6.3

*Always provide function prototypes, even though it's possible to omit them when functions are defined* before *they're used. Providing the prototypes avoids tying the code to the order in which functions are defined (which can easily change as a program evolves).*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.5.1 Function Signatures and Function Prototypes

- The portion of a function prototype that includes the *name of the function* and the *types of its arguments* is called the function signature or simply the signature.
  - Signature does not specify the function's return type.
- The scope of a function is the region of a program in which the function is known and accessible.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Error-Prevention Tip 6.3

*Function prototypes help you find many types of errors at compile time. It's always better to eliminate errors at compile time rather than at run time.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.5.2 Argument Coercion

- An important feature of function prototypes is argument coercion
  - forcing arguments to the appropriate types specified by the parameter declarations.
  - These conversions occur as specified by C++'s promotion rules.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.5.3 Argument-Promotion Rules and Implicit Conversions

- The promotion rules indicate how to convert between types without losing data.
- The promotion rules apply to expressions containing values of two or more data types
  - also referred to as mixed-type expressions.
- The type of each value in a mixed-type expression is promoted to the “highest” type in the expression.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.5.3 Argument-Promotion Rules and Implicit Conversions

- Figure 6.4 lists the arithmetic data types in order from "highest type" to "lowest type."
- Converting values to lower fundamental types can result in incorrect values.
- Therefore, a value can be converted to a lower fundamental type only by explicitly assigning the value to a variable of lower type or by using a cast operator.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

| Data types | |
|---|---|
| `long double` | |
| `double` | |
| `float` | |
| `unsigned long long int` | (synonymous with `unsigned long long`) |
| `long long int` | (synonymous with `long long`) |
| `unsigned long int` | (synonymous with `unsigned long`) |
| `long int` | (synonymous with `long`) |
| `unsigned int` | (synonymous with `unsigned`) |
| `int` | |
| `unsigned short int` | (synonymous with `unsigned short`) |
| `short int` | (synonymous with `short`) |
| `unsigned char` | |
| `char` and `signed char` | |
| `bool` | |

**Fig. 6.4** | Promotion hierarchy for arithmetic data types; the "highest" types are at the top.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Common Programming Error 6.4

*A compilation error occurs if the arguments in a function call cannot be implicitly converted to the expected types specified in the function's prototype.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.6 C++ Standard Library Header Files

- The C++ Standard Library is divided into many portions, each with its own header file.
- The header files contain the function prototypes for the related functions that form each portion of the library.
- The header files also contain definitions of various class types and functions, as well as constants needed by those functions.
- A header file "instructs" the compiler on how to interface with library and user-written components.
- Figure 6.5 lists some common C++ Standard Library header files, most of which are discussed later in the book.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

| Standard Library header | Explanation |
|---|---|
| `<iostream>` | Contains function prototypes for the C++ standard input and output functions, introduced in Chapter 2, and is covered in more detail in Chapter 13, Stream Input/Output: A Deeper Look. |
| `<iomanip>` | Contains function prototypes for stream manipulators that format streams of data. This header is first used in Section 4.10 and is discussed in more detail in Chapter 13, Stream Input/Output: A Deeper Look. |
| `<cmath>` | Contains function prototypes for math library functions (Section 6.3). |
| `<cstdlib>` | Contains function prototypes for conversions of numbers to text, text to numbers, memory allocation, random numbers and various other utility functions. Portions of the header are covered in Section 6.7; Chapter 11, Operator Overloading; Class `string`; Chapter 17, Exception Handling: A Deeper Look; Chapter 22, Bits, Characters, C Strings and `structs`; and Appendix F, C Legacy Code Topics. |
| `<ctime>` | Contains function prototypes and types for manipulating the time and date. This header is used in Section 6.7. |

**Fig. 6.5** | C++ Standard Library headers. (Part 1 of 4.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

| Standard Library header | Explanation |
|---|---|
| `<array>`, `<vector>`, `<list>`, `<forward_list>`, `<deque>`, `<queue>`, `<stack>`, `<map>`, `<unordered_map>`, `<unordered_set>`, `<set>`, `<bitset>` | These headers contain classes that implement the C++ Standard Library containers. Containers store data during a program's execution. The `<vector>` header is first introduced in Chapter 7, Class Templates `array` and `vector`; Catching Exceptions. We discuss all these headers in Chapter 15, Standard Library Containers and Iterators. `<array>`, `<forward_list>`, `<unordered_map>` and `<unordered_set>` were all introduced in C++11. |
| `<cctype>` | Contains function prototypes for functions that test characters for certain properties (such as whether the character is a digit or a punctuation), and function prototypes for functions that can be used to convert lowercase letters to uppercase letters and vice versa. These topics are discussed in Chapter 22, Bits, Characters, C Strings and `structs`. |
| `<cstring>` | Contains function prototypes for C-style string-processing functions. |
| `<typeinfo>` | Contains classes for runtime type identification (determining data types at execution time). This header is discussed in Section 12.9. |

**Fig. 6.5** | C++ Standard Library headers. (Part 2 of 4.)



©1992–2017 by Pearson Education, Inc. All Rights Reserved.

| Standard Library header | Explanation |
|---|---|
| `<exception>`, `<stdexcept>` | These headers contain classes that are used for exception handling (discussed in Chapter 17, Exception Handling: A Deeper Look). |
| `<memory>` | Contains classes and functions used by the C++ Standard Library to allocate memory to the C++ Standard Library containers. This header is used in Chapter 17, Exception Handling: A Deeper Look. |
| `<fstream>` | Contains function prototypes for functions that perform input from and output to files on disk (discussed in Chapter 14, File Processing). |
| `<string>` | Contains the definition of class `string` from the C++ Standard Library (discussed in Chapter 21, Class `string` and String Stream Processing). |
| `<sstream>` | Contains function prototypes for functions that perform input from strings in memory and output to strings in memory (discussed in Chapter 21, Class `string` and String Stream Processing). |
| `<functional>` | Contains classes and functions used by C++ Standard Library algorithms. This header is used in Chapter 16. |
| `<iterator>` | Contains classes for accessing data in the C++ Standard Library containers. This header is used in Chapter 15. |

**Fig. 6.5** | C++ Standard Library headers. (Part 3 of 4.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

| Standard Library header | Explanation |
|---|---|
| `<algorithm>` | Contains functions for manipulating data in C++ Standard Library containers. This header is used in Chapter 15. |
| `<cassert>` | Contains macros for adding diagnostics that aid program debugging. This header is used in Appendix E, Preprocessor. |
| `<cfloat>` | Contains the floating-point size limits of the system. |
| `<climits>` | Contains the integral size limits of the system. |
| `<cstdio>` | Contains function prototypes for the C-style standard input/output library functions. |
| `<locale>` | Contains classes and functions normally used by stream processing to process data in the natural form for different languages (e.g., monetary formats, sorting strings, character presentation, etc.). |
| `<limits>` | Contains classes for defining the numerical data type limits on each computer platform—this is C++'s version of `<climits>` and `<cfloat>`. |
| `<utility>` | Contains classes and functions that are used by many C++ Standard Library headers. |

**Fig. 6.5** | C++ Standard Library headers. (Part 4 of 4.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.7 Case Study: Random Number Generation

- The element of chance can be introduced into computer applications by using the C++ Standard Library function `rand`.
- The function `rand` generates an unsigned integer between 0 and `RAND_MAX` (a symbolic constant defined in the `<cstdlib>` header file).

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.7 Case Study: Random Number Generation

- The value of `RAND_MAX` must be at least 32767—the maximum positive value for a two-byte (16-bit) integer.
- For GNU C++, the value of `RAND_MAX` is 2147483647; for Visual Studio, the value of `RAND_MAX` is 32767.
- If `rand` truly produces integers at random, every number between 0 and `RAND_MAX` has an equal *chance* (or probability) of being chosen each time `rand` is called.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.7.1 Rolling a Six-Sided Die

- The function prototype for the `rand` function is in `<cstdlib>`.
- To produce integers in the range 0 to 5, we use the modulus operator (%) with `rand`:
  - `rand() % 6`
  - This is called scaling.
  - The number 6 is called the scaling factor. Six values are produced.
- We can shift the range of numbers produced by adding a value.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.6: fig06_06.cpp
2  // Shifted, scaled integers produced by 1 + rand() % 6.
3  #include <iostream>
4  #include <iomanip>
5  #include <cstdlib> // contains function prototype for rand
6  using namespace std;
7
8  int main() {
9     // loop 20 times
10    for (unsigned int counter{1}; counter <= 20; ++counter) {
11       // pick random number from 1 to 6 and output it
12       cout << setw(10) << (1 + rand() % 6);
13
14       // if counter is divisible by 5, start a new line of output
15       if (counter % 5 == 0) {
16          cout << endl;
17       }
18    }
19 }
```

```
         6         6         5         5         6
         5         1         1         5         3
         6         6         2         4         2
         6         2         3         4         1
```

**Fig. 6.6** | Shifted, scaled integers produced by `1 + rand() % 6`.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.7.2 Rolling a Six-Sided Die 60,000,000 Times

- To show that the numbers produced by `rand` occur with approximately equal likelihood, Fig. 6.7 simulates 60,000,000 rolls of a die.
- Each integer in the range 1 to 6 should appear approximately 10ccccc,000,000 times.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.7: fig06_07.cpp
2  // Rolling a six-sided die 60,000,000 times.
3  #include <iostream>
4  #include <iomanip>
5  #include <cstdlib> // contains function prototype for rand
6  using namespace std;
7
8  int main() {
9     unsigned int frequency1{0}; // count of 1s rolled
10    unsigned int frequency2{0}; // count of 2s rolled
11    unsigned int frequency3{0}; // count of 3s rolled
12    unsigned int frequency4{0}; // count of 4s rolled
13    unsigned int frequency5{0}; // count of 5s rolled
14    unsigned int frequency6{0}; // count of 6s rolled
15    int face; // stores each roll of the die
16
17    // summarize results of 60,000,000 rolls of a die
18    for (unsigned int roll{1}; roll <= 60'000'000; ++roll) {
19       face = 1 + rand() % 6; // random number from 1 to 6
20
```

**Fig. 6.7** | Rolling a six-sided die 60,000,000 times. (Part 1 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
21          // determine roll value 1-6 and increment appropriate counter
22          switch (face) {
23             case 1:
24                ++frequency1; // increment the 1s counter
25                break;
26             case 2:
27                ++frequency2; // increment the 2s counter
28                break;
29             case 3:
30                ++frequency3; // increment the 3s counter
31                break;
32             case 4:
33                ++frequency4; // increment the 4s counter
34                break;
35             case 5:
36                ++frequency5; // increment the 5s counter
37                break;
38             case 6:
39                ++frequency6; // increment the 6s counter
40                break;
41             default: // invalid value
42                cout << "Program should never get here!";
43          }
44       }
```

**Fig. 6.7** | Rolling a six-sided die 60,000,000 times. (Part 2 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
45
46      cout << "Face" << setw(13) << "Frequency" << endl; // output headers
47      cout << "   1" << setw(13) << frequency1
48         << "\n   2" << setw(13) << frequency2
49         << "\n   3" << setw(13) << frequency3
50         << "\n   4" << setw(13) << frequency4
51         << "\n   5" << setw(13) << frequency5
52         << "\n   6" << setw(13) << frequency6 << endl;
53   }
```

```
Face      Frequency
   1        9999294
   2       10002929
   3        9995360
   4       10000409
   5       10005206
   6        9996802
```

**Fig. 6.7** | Rolling a six-sided die 60,000,000 times. (Part 3 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Error-Prevention Tip 6.4

*Provide a* `default` *case in a* `switch` *to catch errors even if you are absolutely, positively certain that you have no bugs!*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.7.2 Rolling a Six-Sided Die 60,000,000 Times

- Prior to C++14, you'd represent the integer value 60,000,000 as 60000000 in a program.
- To make numeric literals more readable, C++14 allows you to insert between groups of digits in numeric literals the digit separator ' (a single-quote character)
  - 60'000'000 represents the integer value 60,000,000.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.7.3 Randomizing the Random-Number Generator with `srand`

- Executing the program of Fig. 6.6 again produces exactly the same sequence of values.
  - Repeatability is an important characteristic of `rand`.
  - Can help with testing and debugging.
- `rand` generates pseudorandom numbers.
  - a sequence of numbers that appears to be random.
  - sequence repeats itself each time the program executes.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Error-Prevention Tip 6.5

*When debugging a simulation program, random-number repeatability is essential for proving that corrections to the program work properly.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.7.3 Randomizing the Random-Number Generator with `srand`

- Once a program has been debugged, it can be conditioned to produce a different sequence of random numbers for each execution.
- This is called randomizing and is accomplished with the C++ Standard Library function `srand`.
  - Takes an `unsigned` integer argument and seeds `rand` to produce a different sequence of random numbers for each execution.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.7 Case Study: Random Number Generation (cont.)

***Seeding the Random Number Generator with srand***

- Figure 6.8 demonstrates function `srand`.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Software Engineering Observation 6.4

*Ensure that your program seeds the random-number generator* differently *(and* only once*) each time the program executes; otherwise, an attacker would easily be able to determine the sequence of pseudorandom numbers that would be produced.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
1  // Fig. 6.8: fig06_08.cpp
2  // Randomizing the die-rolling program.
3  #include <iostream>
4  #include <iomanip>
5  #include <cstdlib> // contains prototypes for functions srand and rand
6  using namespace std;
7
8  int main() {
9     unsigned int seed{0}; // stores the seed entered by the user
10
11    cout << "Enter seed: ";
12    cin >> seed;
13    srand(seed); // seed random number generator
14
```

**Fig. 6.8** | Randomizing the die-rolling program. (Part 1 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
15      // loop 10 times
16      for (unsigned int counter{1}; counter <= 10; ++counter) {
17         // pick random number from 1 to 6 and output it
18         cout << setw(10) << (1 + rand() % 6);
19
20         // if counter is divisible by 5, start a new line of output
21         if (counter % 5 == 0) {
22            cout << endl;
23         }
24      }
25   }
```

**Fig. 6.8** | Randomizing the die-rolling program. (Part 2 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
Enter seed: 67
        6       1       4       6       2
        1       6       1       6       4
```

```
Enter seed: 432
        4       6       3       1       6
        3       1       5       4       2
```

```
Enter seed: 67
        6       1       4       6       2
        1       6       1       6       4
```

**Fig. 6.8** | Randomizing the die-rolling program. (Part 3 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.7.4 Seeding the Random-Number Generator with the Current Time

- To randomize *without* having to enter a seed
  - `srand(static_cast<unsigned int>(time(0)));`
- Causes the computer to read its *clock* to obtain the value for the seed.
- Function `time` (with the argument `0` as written in the preceding statement) returns the current time as the number of seconds since January 1, 1970, at midnight Greenwich Mean Time (GMT).
- The function prototype for `time` is in `<ctime>`.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.7.5 Scaling and Shifting Random Numbers

- To produce random numbers in a specific range use:
  - *type variableName* = *shiftingValue* + `rand()` % *scalingFactor*;
- *shiftingValue* is equal to the *first number* in the desired range of consecutive integers
- *scalingFactor* is equal to the *width* of the de-sired range of consecutive integers.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.8 Case Study: Game of Chance; Introducing enum

- Rules of Craps
  - A player rolls two dice. Each die has six faces.
  - Faces contain 1, 2, 3, 4, 5 and 6 spots.
  - After the dice have come to rest, the sum of the spots on the two upward faces is calculated.
    - If the sum is 7 or 11 on the first roll, the player wins.
    - If the sum is 2, 3 or 12 on the first roll (called "craps"), the player loses (i.e., the "house" wins).
    - If the sum is 4, 5, 6, 8, 9 or 10 on the first roll, then that sum becomes the player's "point."
  - To win, continue rolling until you "make your point."
  - You lose by rolling a 7 before making the point.
- The program in Fig. 6.9 simulates the game.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.9: fig06_09.cpp
2  // Craps simulation.
3  #include <iostream>
4  #include <cstdlib> // contains prototypes for functions srand and rand
5  #include <ctime> // contains prototype for function time
6  using namespace std;
7
8  unsigned int rollDice(); // rolls dice, calculates and displays sum
9
10 int main() {
11    // scoped enumeration with constants that represent the game status
12    enum class Status {CONTINUE, WON, LOST}; // all caps in constants
13
14    // randomize random number generator using current time
15    srand(static_cast<unsigned int>(time(0)));
16
17    unsigned int myPoint{0}; // point if no win or loss on first roll
18    Status gameStatus; // can be CONTINUE, WON or LOST
19    unsigned int sumOfDice{rollDice()}; // first roll of the dice
20
```

**Fig. 6.9** | Craps simulation. (Part 1 of 5.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
21      // determine game status and point (if needed) based on first roll
22      switch (sumOfDice) {
23         case 7: // win with 7 on first roll
24         case 11: // win with 11 on first roll
25            gameStatus = Status::WON;
26            break;
27         case 2: // lose with 2 on first roll
28         case 3: // lose with 3 on first roll
29         case 12: // lose with 12 on first roll
30            gameStatus = Status::LOST;
31            break;
32         default: // did not win or lose, so remember point
33            gameStatus = Status::CONTINUE; // game is not over
34            myPoint = sumOfDice; // remember the point
35            cout << "Point is " << myPoint << endl;
36            break; // optional at end of switch
37      }
38
```

**Fig. 6.9** | Craps simulation. (Part 2 of 5.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
39      // while game is not complete
40      while (Status::CONTINUE == gameStatus) { // not WON or LOST
41         sumOfDice = rollDice(); // roll dice again
42
43         // determine game status
44         if (sumOfDice == myPoint) { // win by making point
45            gameStatus = Status::WON;
46         }
47         else {
48            if (sumOfDice == 7) { // lose by rolling 7 before point
49               gameStatus = Status::LOST;
50            }
51         }
52      }
53
54      // display won or lost message
55      if (Status::WON == gameStatus) {
56         cout << "Player wins" << endl;
57      }
58      else {
59         cout << "Player loses" << endl;
60      }
61   }
62
```

**Fig. 6.9** | Craps simulation. (Part 3 of 5.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
63  // roll dice, calculate sum and display results
64  unsigned int rollDice() {
65     int die1{1 + rand() % 6}; // first die roll
66     int die2{1 + rand() % 6}; // second die roll
67     int sum{die1 + die2}; // compute sum of die values
68
69     // display results of this roll
70     cout << "Player rolled " << die1 << " + " << die2
71        << " = " << sum << endl;
72     return sum;
73  }
```

```
Player rolled 2 + 5 = 7
Player wins
```

```
Player rolled 6 + 6 = 12
Player loses
```

**Fig. 6.9** | Craps simulation. (Part 4 of 5.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
Player rolled 1 + 3 = 4
Point is 4
Player rolled 4 + 6 = 10
Player rolled 2 + 4 = 6
Player rolled 6 + 4 = 10
Player rolled 2 + 3 = 5
Player rolled 2 + 4 = 6
Player rolled 1 + 1 = 2
Player rolled 4 + 4 = 8
Player rolled 4 + 3 = 7
Player loses
```

```
Player rolled 3 + 3 = 6
Point is 6
Player rolled 5 + 3 = 8
Player rolled 4 + 5 = 9
Player rolled 2 + 1 = 3
Player rolled 1 + 5 = 6
Player wins
```

**Fig. 6.9** | Craps simulation. (Part 5 of 5.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.8 Case Study: Game of Chance; Introducing `enum` (cont.)

- A scoped enumeration (C++11) introduced by `enum class` and followed by a type name, is a set of identifiers representing integer constants.
- The values of these enumeration constants start at `0`, unless specified otherwise, and increment by `1`.
- The identifiers must be unique, but separate enumeration constants can have the same value.
- Variables of an `enum class` can be assigned only one of the values declared in the enumeration.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Good Programming Practice 6.2

*By convention, you should capitalize the first letter of an* `enum` `class`*'s name.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Good Programming Practice 6.3

*Use only uppercase letters in enumeration constant names. This makes these constants stand out in a program and reminds you that enumeration constants are* not *variables.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.8 Case Study: Game of Chance; Introducing `enum` (cont.)

- To reference a scoped `enum` constant, qualify it with the type name and the scope-resolution operator (`::`), as in `Status::CONTINUE`.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Error-Prevention Tip 6.6

*Qualifying an* `enum class`*'s constant with its typename and* `::` *explicitly identifies the constant as being in the scope of the specified* `enum class`*. If another* `enum class` *contains the same identifier for one of its constants, it's always clear which version of the constant is being used, because the typename and* `::` *are required.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.8 Case Study: Game of Chance; Introducing enum (cont.)

- Another popular enumeration is
  - `enum class Months { JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC };`
  - creates user-defined type `Months` with enumeration constants representing the months of the year.
  - The first value in the preceding enumeration is explicitly set to 1, so the remaining values increment from 1, resulting in the values 1 through 12.
- Any enumeration constant can be assigned an integer value in the enumeration definition.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Error-Prevention Tip 6.7

*Use unique values for an* `enum`*'s constants to help prevent hard-to-find logic errors.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.8 Case Study: Game of Chance; Introducing enum (cont.)

- Prior to C++11, enumerations were defined with `enum` (unscoped `enum`s)
- Problem: multiple `enum`s may contain the same identifiers.
- Using such `enum`s in the same program can lead to naming collisions and logic errors.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Error-Prevention Tip 6.8t

*Use scoped* `enum`*s to avoid the potential naming conflicts that can occur with unscoped* `enum` *constants.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.8 Case Study: Game of Chance; Introducing enum (cont.)

- Enumeration constants are represented as integers.
- An unscoped `enum`'s underlying integral type depends on its constants' values—the type is guaranteed to be large enough to store the constant values specified.
- A scoped `enum`'s underlying integral type is `int`.
- You to specify a scoped `enum`'s underlying integral type by following the `enum`'s type name with a colon (`:`) and the integral type.
- We can specify that the constants in the `enum class Status` should have type `unsigned int`, as in
  - `enum class Status : unsigned int {CONTINUE, WON, LOST};`

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Common Programming Error 6.5

*A compilation error occurs if an* `enum` *constant's value is outside the range that can be represented by the* `enum`*'s underlying type.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.9 C++11 Random Numbers

- According to CERT, function `rand` does not have "good statistical properties" and can be predictable, which makes programs that use rand less secure (CERT guideline MSC30-CPP).
- C++11 provides a new, *more secure* library of random-number capabilities that can produce nondeterministic random numbers that can't be predicted.
- Such random-number generators are used in simulations and security scenarios where predictability is undesirable.
  - New capabilities are located in the C++ Standard Library's `<random>` header.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.9 C++11 Random Numbers (cont.)

- For flexibility based on how random numbers are used in programs, C++11 provides many classes that represent various random-number generation *engines* and *distributions*.
  - An engine implements a random-number generation algorithm that produce pseudorandom numbers.
  - A distribution controls the range of values produced by an engine, the types of those values (e.g., int, double, etc.) and the statistical properties of the values.
- `uniform_int_distribution` evenly distributes pseudorandom integers over a specified range of values.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.9 C++11 Random Numbers (cont.)

***Rolling a Six-Sided Die***

- Figure 6.10 uses the `default_random_engine` and the `uniform_int_distribution` to roll a six-sided die.
  - `default_random_engine` object named `engine`.
  - Its constructor argument *seeds* the random-number generation engine with the current time.
  - If you don't pass a value to the constructor, the default seed will be used and the program will produce the *same* sequence of numbers each time it executes.
  - `randomInt`—a `uniform_int_distribution` object that produces `unsigned int` values (as specified by `<unsigned int>`) in the range 1 to 6 (as specified by the constructor arguments).
  - The expression `randomInt(engine)` returns one `unsigned int` value in the range 1 to 6.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.9 C++11 Random Numbers (cont.)

- The notation `<unsigned int>` indicates that `uniform_int_distribution` is a *class template*.
- In this case, any integer type can be specified in the angle brackets (< and >).
- In Chapter 18, we discuss how to create class templates and various other chapters show how to use existing class templates from the C++ Standard Library.
- For now, you should feel comfortable using class template `uniform_int_distribution` by mimicking the syntax shown in the example.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.10: fig06_10.cpp
2  // Using a C++11 random-number generation engine and distribution
3  // to roll a six-sided die.
4  #include <iostream>
5  #include <iomanip>
6  #include <random> // contains C++11 random number generation features
7  #include <ctime>
8  using namespace std;
9
10 int main() {
11    // use the default random-number generation engine to
12    // produce uniformly distributed pseudorandom int values from 1 to 6
13    default_random_engine engine{static_cast<unsigned int>(time(0))};
14    uniform_int_distribution<unsigned int> randomInt{1, 6};
15
```

**Fig. 6.10** | Using a C++11 random-number generation engine and distribution to roll a six-sided die. (Part 1 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
16      // loop 10 times
17      for (unsigned int counter{1}; counter <= 10; ++counter) {
18         // pick random number from 1 to 6 and output it
19         cout << setw(10) << randomInt(engine);
20
21         // if counter is divisible by 5, start a new line of output
22         if (counter % 5 == 0) {
23            cout << endl;
24         }
25      }
26   }
```

```
         2         1         2         3         5
         6         1         5         6         4
```

**Fig. 6.10** | Using a C++11 random-number generation engine and distribution to roll a six-sided die. (Part 2 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.10 Scope Rules

- The portion of the program where an identifier can be used is known as its *scope*.
- This section discusses
  - block scope
  - global namespace scope

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.10 Scope Rules (cont.)

- Identifiers declared *inside* a block have *block scope*, which begins at the identifier's declaration and ends at the terminating right brace (}) of the enclosing block.
  - Local variables have block scope, as do function parameters.
- Any block can contain variable declarations.
- In nested blocks, if an identifier in an outer block has the same name as an identifier in an inner block, the one in the outer block is "hidden" until the inner block terminates.
- The inner block "sees" its own local variable's value and not that of the enclosing block's identically named variable.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Common Programming Error 6.6

*Accidentally using the same name for an identifier in an inner block that's used for an identifier in an outer block, when in fact you want the identifier in the outer block to be active for the duration of the inner block, is typically a logic error.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Error-Prevention Tip 6.9

*Avoid variable names in inner scopes that hide names in outer scopes. Most compilers will warn you about this issue.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.10 Scope Rules (cont.)

- An identifier declared outside any function or class has *global namespace scope*.
  - "known" in all functions from the point at which it's declared until the end of the file.
- Function definitions, function prototypes placed outside a function, class definitions and global variables all have global namespace scope.
- Global variables are created by placing variable declarations outside any class or function definition. Such variables retain their values throughout a program's execution.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Software Engineering Observation 6.5

*Declaring a variable as global rather than local allows* unintended side effects *to occur when a function that does not need access to the variable accidentally or maliciously modifies it. This is another example of the principle of least privilege—except for truly global resources such as* `cin` *and* `cout`*, global variables should be avoided. In general, variables should be declared in the narrowest scope in which they need to be accessed.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Software Engineering Observation 6.6

*Variables used only in a particular function should be declared as local variables in that function rather than as global variables.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.10 Scope Rules (cont.)

***Scope Demonstration***

- The program of Fig. 6.11 demonstrates scoping issues with global variables, automatic local variables and `static` local variables.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.11: fig06_11.cpp
2  // Scoping example.
3  #include <iostream>
4  using namespace std;
5
6  void useLocal(); // function prototype
7  void useStaticLocal(); // function prototype
8  void useGlobal(); // function prototype
9
10 int x{1}; // global variable
11
12 int main() {
13    cout << "global x in main is " << x << endl;
14
15    int x{5}; // local variable to main
16
17    cout << "local x in main's outer scope is " << x << endl;
18
19    { // block starts a new scope
20       int x{7}; // hides both x in outer scope and global x
21
22       cout << "local x in main's inner scope is " << x << endl;
23    }
24
```

**Fig. 6.11** | Scoping example. (Part 1 of 4.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
25      cout << "local x in main's outer scope is " << x << endl;
26
27      useLocal(); // useLocal has local x
28      useStaticLocal(); // useStaticLocal has static local x
29      useGlobal(); // useGlobal uses global x
30      useLocal(); // useLocal reinitializes its local x
31      useStaticLocal(); // static local x retains its prior value
32      useGlobal(); // global x also retains its prior value
33
34      cout << "\nlocal x in main is " << x << endl;
35  }
36
37  // useLocal reinitializes local variable x during each call
38  void useLocal() {
39      int x{25}; // initialized each time useLocal is called
40
41      cout << "\nlocal x is " << x << " on entering useLocal" << endl;
42      ++x;
43      cout << "local x is " << x << " on exiting useLocal" << endl;
44  }
45
```

**Fig. 6.11** | Scoping example. (Part 2 of 4.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
1  // useStaticLocal initializes static local variable x only the
2  // first time the function is called; value of x is saved
3  // between calls to this function
4  void useStaticLocal() {
5     static int x{50}; // initialized first time useStaticLocal is called
6
7     cout << "\nlocal static x is " << x << " on entering useStaticLocal"
8        << endl;
9     ++x;
10    cout << "local static x is " << x << " on exiting useStaticLocal"
11       << endl;
12 }
13
14 // useGlobal modifies global variable x during each call
15 void useGlobal() {
16    cout << "\nglobal x is " << x << " on entering useGlobal" << endl;
17    x *= 10;
18    cout << "global x is " << x << " on exiting useGlobal" << endl;
19 }
```

**Fig. 6.11** | Scoping example. (Part 3 of 4.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
global x in main is 1
local x in main's outer scope is 5
local x in main's inner scope is 7
local x in main's outer scope is 5

local x is 25 on entering useLocal
local x is 26 on exiting useLocal

local static x is 50 on entering useStaticLocal
local static x is 51 on exiting useStaticLocal

global x is 1 on entering useGlobal
global x is 10 on exiting useGlobal

local x is 25 on entering useLocal
local x is 26 on exiting useLocal

local static x is 51 on entering useStaticLocal
local static x is 52 on exiting useStaticLocal

global x is 10 on entering useGlobal
global x is 100 on exiting useGlobal

local x in main is 5
```

**Fig. 6.11** | Scoping example. (Part 4 of 4.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.11 Function Call Stack and Activation Records

- To understand how C++ performs function calls, we first need to consider a data structure (i.e., collection of related data items) known as a stack.
- Analogous to a pile of dishes.
  - When a dish is placed on the pile, it's normally placed at the *top*—referred to as pushing.
  - Similarly, when a dish is removed from the pile, it's normally removed from the top—referred to as popping.
- Last-in, first-out (LIFO) data structures—the last item pushed (inserted) is the first item popped (removed).

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.11 Function Call Stack and Activation Records (cont.)

***Function-Call Stack***

- One of the most important mechanisms for computer science students to understand is the function call stack (or program execution stack).
  - supports the function call/return mechanism.
  - Also supports the creation, maintenance and destruction of each called function's automatic variables.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.11 Function Call Stack and Activation Records (cont.)

***Stack Frames***

- Each function eventually must return control to the function that called it.
- Each time a function calls another function, an entry is pushed onto the function call stack.
  - This entry, called a stack frame or an activation record, contains the *return address* that the called function needs in order to return to the calling function.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.11 Function Call Stack and Activation Records (cont.)

***Automatic Local Variables and Stack Frames***

- When a function call returns, the stack frame for the function call is *popped*, and control transfers to the return address in the popped stack frame.

***Stack Overflow***

- The amount of memory in a computer is finite, so only a certain amount of memory can be used to store activation records on the function call stack.
- If more function calls occur than can have their activation records stored on the function call stack, an a fatal error known as stack overflow occurs.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.11 Function Call Stack and Activation Records (cont.)

***Function Call Stack in Action***

- Now let's consider how the call stack supports the operation of a square function called by main (Fig. 6.12).
- First the operating system calls main—this pushes an activation record onto the stack (shown in Fig. 6.13).
- The activation record tells main how to return to the operating system (i.e., transfer to return address R1) and contains the space for main's automatic variable (i.e., a, which is initialized to 10).

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.12: fig06_12.cpp
2  // square function used to demonstrate the function
3  // call stack and activation records.
4  #include <iostream>
5  using namespace std;
6
7  int square(int); // prototype for function square
8
9  int main() {
10    int a{10}; // value to square (local variable in main)
11
12    cout << a << " squared: " << square(a) << endl; // display a squared
13 }
14
15 // returns the square of an integer
16 int square(int x) { // x is a local variable
17    return x * x; // calculate square and return result
18 }
```

```
10 squared: 100
```

**Fig. 6.12** | square function used to demonstrate the function-call stack and activation records.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

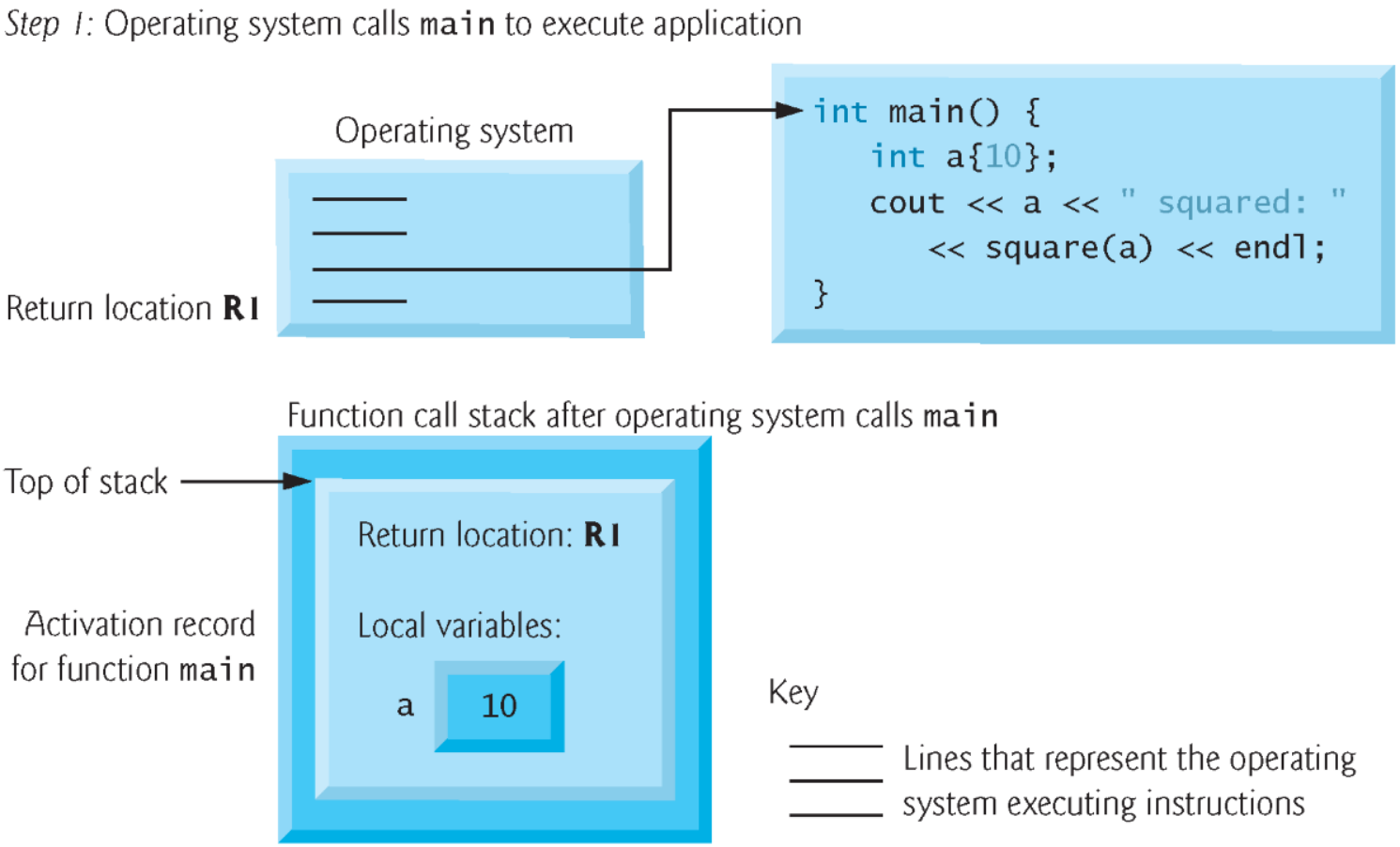



**Fig. 6.13** | Function-call stack after the operating system calls `main` to execute the program.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.12 Function Call Stack and Activation Records (cont.)

- Function `main`—before returning to the operating system—now calls function `square` in line 12 of Fig. 6.12.
- This causes a stack frame for `square` (lines 16–18) to be pushed onto the function call stack (Fig. 6.14).
- This stack frame contains the return address that `square` needs to return to `main` (i.e., R2) and the memory for `square`'s automatic variable (i.e., `x`).

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

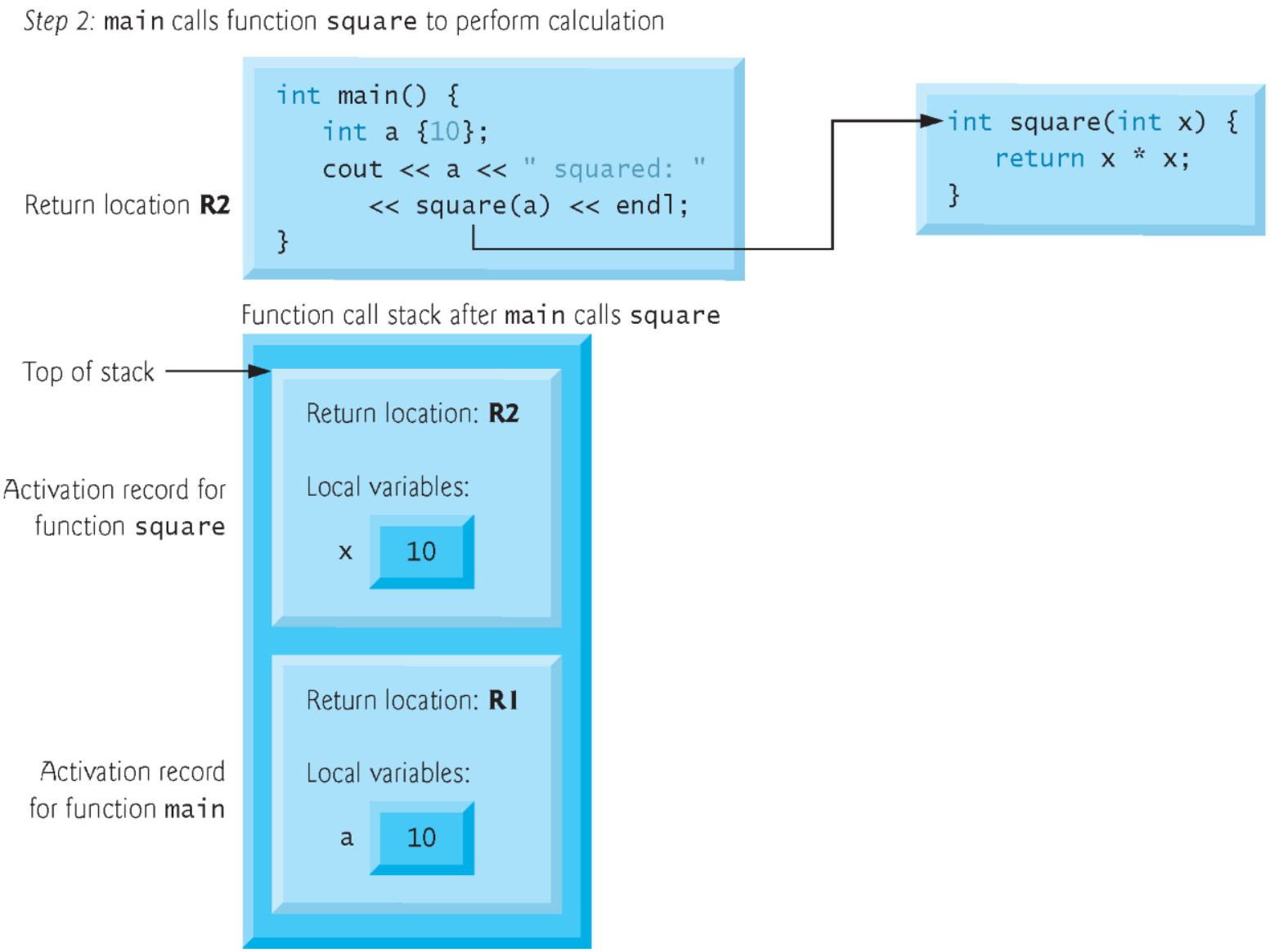



**Fig. 6.14** | Function-call stack after `main` calls `square` to perform the calculation.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.12 Function Call Stack and Activation Records (cont.)

- After `square` calculates the square of its argument, it needs to return to `main`—and no longer needs the memory for its automatic variable `x`. So `square`'s stack frame is *popped* from the stack—giving `square` the return location in `main` (i.e., R2) and losing `square`'s automatic variable.
- Figure 6.15 shows the function call stack *after* `square`'s activation record has been popped.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

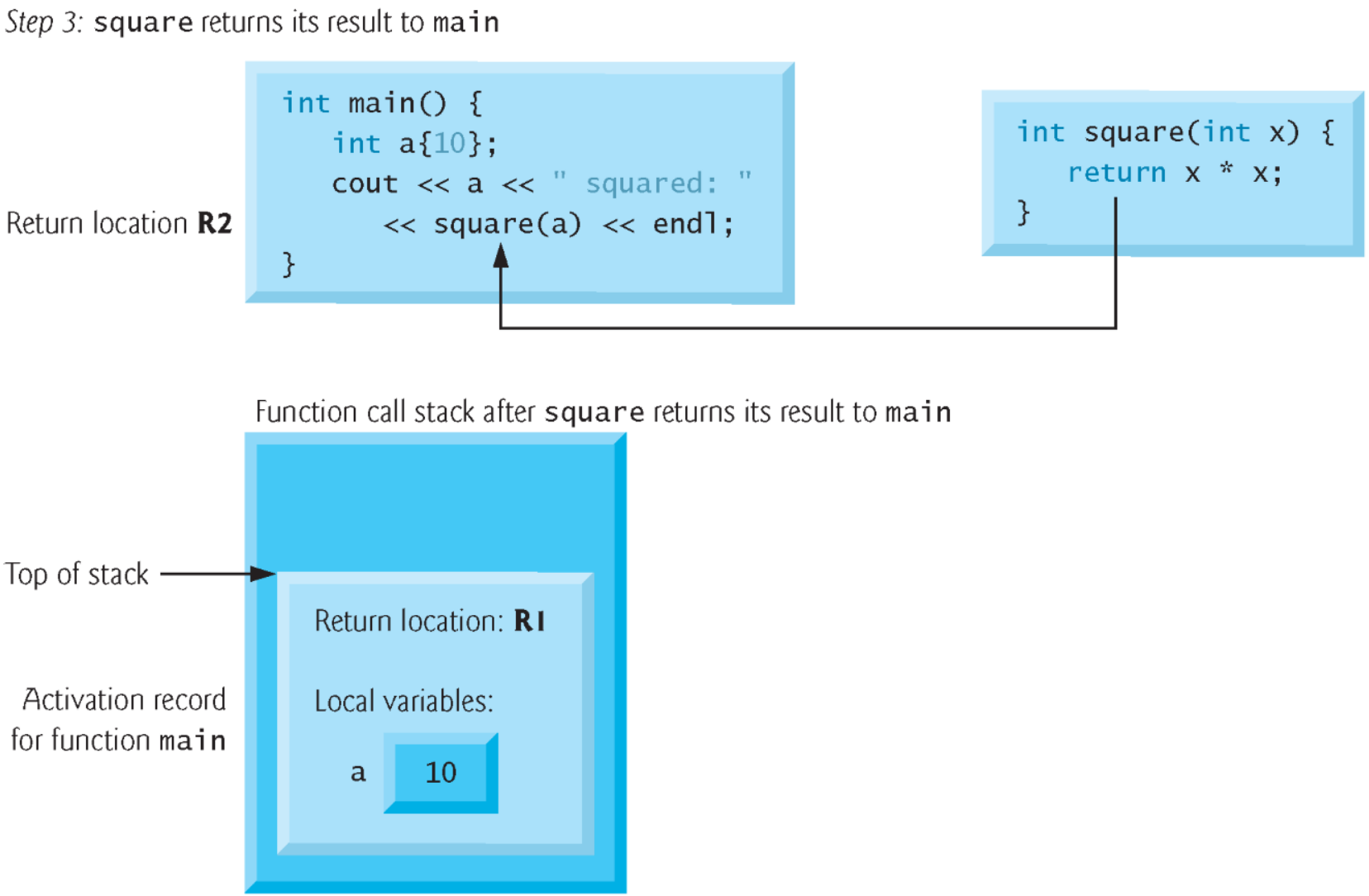



**Fig. 6.15** | Function-call stack after function `square` returns to `main`.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.12 Inline Functions

- C++ provides inline functions to help reduce function call overhead.
- Placing the qualifier `inline` before a function's return type in the function definition *advises* the compiler to generate a copy of the function's code in *every* place where the function is called (when appropriate) to avoid a function call.
- This often makes the program *larger*.
- Reusable `inline` functions are typically placed in headers, so that their definitions can be included in each source file that uses them.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



**Software Engineering Observation 6.7**

*If you change the definition of an* `inline` *function, you must recompile all of that function's clients.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Performance Tip 6.1

*Compilers can inline code for which you have not explicitly used the* `inline` *keyword. Today's optimizing compilers are so sophisticated that it's best to leave inlining decisions to the compiler.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.14 Inline Functions

- Figure 6.16 uses inline function `cube` (lines 9–11) to calculate the volume of a cube.
- Keyword `const` in function `cube`'s parameter list (line 9) tells the compiler that the function does not modify variable `side`.
- This ensures that `side`'s value is *not* changed by the function during the calculation.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Software Engineering Observation 6.8

*The* `const` *qualifier should be used to enforce the principle of least privilege. Using this principle to properly design software can greatly reduce debugging time and improper side effects and can make a program easier to modify and maintain.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
1  // Fig. 6.16: fig06_16.cpp
2  // inline function that calculates the volume of a cube.
3  #include <iostream>
4  using namespace std;
5
6  // Definition of inline function cube. Definition of function appears
7  // before function is called, so a function prototype is not required.
8  // First line of function definition acts as the prototype.
9  inline double cube(const double side) {
10    return side * side * side; // calculate cube
11 }
12
13 int main() {
14    double sideValue; // stores value entered by user
15    cout << "Enter the side length of your cube: ";
16    cin >> sideValue; // read value from user
17
18    // calculate cube of sideValue and display result
19    cout << "Volume of cube with side "
20       << sideValue << " is " << cube(sideValue) << endl;
21 }
```

```
Enter the side length of your cube: 3.5
Volume of cube with side 3.5 is 42.875
```

**Fig. 6.16** | `inline` function that calculates the volume of a cube.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.13 References and Reference Parameters

- Two ways to pass arguments to functions in many programming languages are pass-by-value and pass-by-reference.
- When an argument is passed by value, a *copy* of the argument's value is made and passed (on the function call stack) to the called function.
  - Changes to the copy do not affect the original variable's value in the caller.
- To specify a reference to a constant, place the `const` qualifier before the type specifier in the parameter declaration.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Performance Tip 6.2

*One disadvantage of pass-by-value is that, if a large data item is being passed, copying that data can take a considerable amount of execution time and memory space.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.15 References and Reference Parameters (cont.)

- With *pass-by-reference*, the caller gives the called function the ability to *access the caller's data directly*, and to *modify* that data.
- A reference parameter is an *alias* for its corresponding argument in a function call.
- To indicate that a function parameter is passed by reference, simply follow the parameter's type in the function prototype by an *ampersand* (&); use the same convention when listing the parameter's type in the function header.
- Figure 6.17 compares pass-by-value and pass-by-reference with reference parameters.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Performance Tip 6.3

*Pass-by-reference is good for performance reasons, because it can eliminate the pass-by-value overhead of copying large amounts of data.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Software Engineering Observation 6.9

*Pass-by-reference can weaken security; the called function can corrupt the caller's data.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Common Programming Error 6.7

*Because reference parameters are mentioned only by name in the body of the called function, you might inadvertently treat reference parameters as pass-by-value parameters. This can cause unexpected side effects if the original variables are changed by the function.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
1  // Fig. 6.17: fig06_17cpp
2  // Passing arguments by value and by reference.
3  #include <iostream>
4  using namespace std;
5
6  int squareByValue(int); // function prototype (value pass)
7  void squareByReference(int&); // function prototype (reference pass)
8
9  int main() {
10    int x{2}; // value to square using squareByValue
11    int z{4}; // value to square using squareByReference
12
13    // demonstrate squareByValue
14    cout << "x = " << x << " before squareByValue\n";
15    cout << "Value returned by squareByValue: "
16       << squareByValue(x) << endl;
17    cout << "x = " << x << " after squareByValue\n" << endl;
18
19    // demonstrate squareByReference
20    cout << "z = " << z << " before squareByReference" << endl;
21    squareByReference(z);
22    cout << "z = " << z << " after squareByReference" << endl;
23 }
24
```

**Fig. 6.17** | Passing arguments by value and by reference. (Part 1 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
25  // squareByValue multiplies number by itself, stores the
26  // result in number and returns the new value of number
27  int squareByValue(int number) {
28     return number *= number; // caller's argument not modified
29  }
30
31  // squareByReference multiplies numberRef by itself and stores the result
32  // in the variable to which numberRef refers in function main
33  void squareByReference(int& numberRef) {
34     numberRef *= numberRef; // caller's argument modified
35  }
```

```
x = 2 before squareByValue
Value returned by squareByValue: 4
x = 2 after squareByValue

z = 4 before squareByReference
z = 16 after squareByReference
```

**Fig. 6.17** | Passing arguments by value and by reference. (Part 2 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.15 References and Reference Parameters (cont.)

- References can also be used as aliases for other variables *within* a function.
- Reference variables must be initialized in their declarations and cannot be reassigned as *aliases* to other variables.
- Once a reference is declared as an alias for another variable, all operations supposedly performed on the alias are actually performed on the *original* variable.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.15 References and Reference Parameters (cont.)

- To specify that a reference parameter should not be allowed to modify the corresponding argument, place the `const` qualifier before the type name in the parameter's declaration.
- `string` objects can be large, so they (and objects in general) should be passed to functions by reference.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Performance Tip 6.4

*For passing large objects, use a* `const` *reference parameter to simulate the appearance and security of pass-by-value and avoid the overhead of passing a copy of the large object.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.15 References and Reference Parameters (cont.)

- Functions can return references, but this can be dangerous.
- When returning a reference to a variable declared in the called function, the variable should be declared `static` in that function.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Common Programming Error 6.8

*Returning a reference to a local variable in a called function is a logic error for which compilers typically issue a warning. Compilation warnings indicate potential problems, so most software-engineering teams have policies requiring code to compile without warnings.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.14 Default Arguments

- It common for a program to invoke a function repeatedly with the *same* argument value for a particular parameter.
- Can specify that such a parameter has a default argument, i.e., a default value to be passed to that parameter.
- When a program omits an argument for a parameter with a default argument in a function call, the compiler rewrites the function call and inserts the default value of that argument.
- Figure 6.18 demonstrates using default arguments to calculate a box's volume.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.18: fig06_18.cpp
2  // Using default arguments.
3  #include <iostream>
4  using namespace std;
5
6  // function prototype that specifies default arguments
7  unsigned int boxVolume(unsigned int length = 1, unsigned int width = 1,
8     unsigned int height = 1);
9
10 int main() {
11    // no arguments--use default values for all dimensions
12    cout << "The default box volume is: " << boxVolume();
13
14    // specify length; default width and height
15    cout << "\n\nThe volume of a box with length 10,\n"
16       << "width 1 and height 1 is: " << boxVolume(10);
17
18    // specify length and width; default height
19    cout << "\n\nThe volume of a box with length 10,\n"
20       << "width 5 and height 1 is: " << boxVolume(10, 5);
21
```

**Fig. 6.18** | Using default arguments. (Part 1 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
22      // specify all arguments
23      cout << "\n\nThe volume of a box with length 10,\n"
24         << "width 5 and height 2 is: " << boxVolume(10, 5, 2)
25         << endl;
26   }
27
28   // function boxVolume calculates the volume of a box
29   unsigned int boxVolume(unsigned int length, unsigned int width,
30      unsigned int height) {
31      return length * width * height;
32   }
```

```
The default box volume is: 1

The volume of a box with length 10,
width 1 and height 1 is: 10

The volume of a box with length 10,
width 5 and height 1 is: 50

The volume of a box with length 10,
width 5 and height 2 is: 100
```

**Fig. 6.18** | Using default arguments. (Part 2 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.14 Default Arguments

- Default arguments must be the rightmost (trailing) arguments in a function's parameter list.
- When calling a function with two or more default arguments, if an omitted argument is not the rightmost argument, then all arguments to the right of that argument also must be omitted.
- Default arguments must be specified with the first occurrence of the function name—typically, in the function prototype.
- Default values can be any expression, including constants, global variables or function calls. Default arguments also can be used with inline functions.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Good Programming Practice 6.4

*Using default arguments can simplify writing function calls. However, some programmers feel that explicitly specifying all arguments is clearer.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.15 Unary Scope Resolution Operator

- C++ provides the unary scope resolution operator (`::`) to access a global variable when a local variable of the same name is in scope.
- Using the unary scope resolution operator (`::`) with a given variable name is optional when the only variable with that name is a global variable.
- Figure 6.19 shows the unary scope resolution operator with local and global variables of the same name.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.19: fig06_19.cpp
2  // Unary scope resolution operator.
3  #include <iostream>
4  using namespace std;
5
6  int number{7}; // global variable named number
7
8  int main() {
9     double number{10.5}; // local variable named number
10
11    // display values of local and global variables
12    cout << "Local double value of number = " << number
13       << "\nGlobal int value of number = " << ::number << endl;
14 }
```

```
Local double value of number = 10.5
Global int value of number = 7
```

**Fig. 6.19** | Unary scope resolution operator.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Good Programming Practice 6.5

*Always using the unary scope resolution operator (`::`) to refer to global variables (even if there is no collision with a local-variable name) makes it clear that you're intending to access a global variable rather than a local variable.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Software Engineering Observation 6.10

*Always using the unary scope resolution operator (`::`) to refer to global variables makes programs easier to modify by reducing the risk of name collisions with nonglobal variables.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Error-Prevention Tip 6.10

*Always using the unary scope resolution operator (`::`) to refer to a global variable eliminates logic errors that might occur if a nonglobal variable hides the global variable.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Error-Prevention Tip 6.11

*Avoid using variables of the same name for different purposes in a program. Although this is allowed in various circumstances, it can lead to errors.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.16 Function Overloading

- C++ enables several functions of the same name to be defined, as long as they have different signatures.
- This is called function overloading.
- The C++ compiler selects the proper function to call by examining the number, types and order of the arguments in the call.
- Function overloading is used to create several functions of the *same* name that perform similar tasks, but on different data types.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Good Programming Practice 6.6

*Overloading functions that perform closely related tasks can make programs more readable and understandable.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.16 Function Overloading (cont.)

- Figure 6.20 uses overloaded square functions to calculate the square of an `int` and the square of a `double`.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.20: fig06_20.cpp
2  // Overloaded square functions.
3  #include <iostream>
4  using namespace std;
5
6  // function square for int values
7  int square(int x) {
8     cout << "square of integer " << x << " is ";
9     return x * x;
10 }
11
12 // function square for double values
13 double square(double y) {
14    cout << "square of double " << y << " is ";
15    return y * y;
16 }
17
18 int main() {
19    cout << square(7); // calls int version
20    cout << endl;
21    cout << square(7.5); // calls double version
22    cout << endl;
23 }
```

**Fig. 6.20** | Overloaded `square` functions. (Part 1 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
square of integer 7 is 49
square of double 7.5 is 56.25
```

**Fig. 6.20** | Overloaded `square` functions. (Part 2 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.16 Function Overloading (cont.)

***How the Compiler Differentiates Among Overloaded Functions***

- Overloaded functions are distinguished by their signatures.
- A signature is a combination of a function's name and its parameter types (in order).
- The compiler encodes each function identifier with the types of its parameters (sometimes referred to as name mangling or name decoration) to enable type-safe linkage.
  - Ensures that the proper overloaded function is called and that the types of the arguments conform to the types of the parameters.
- Figure 6.21 was compiled with GNU C++.
- Rather than showing the execution output of the program, we show the mangled function names produced in assembly language by GNU C++.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.21: fig06_21.cpp
2  // Name mangling to enable type-safe linkage.
3
4  // function square for int values
5  int square(int x) {
6     return x * x;
7  }
8
9  // function square for double values
10 double square(double y) {
11    return y * y;
12 }
13
14 // function that receives arguments of types
15 // int, float, char and int&
16 void nothing1(int a, float b, char c, int& d) { }
17
18 // function that receives arguments of types
19 // char, int, float& and double&
20 int nothing2(char a, int b, float& c, double& d) {
21    return 0;
22 }
23
24 int main() { }
```

**Fig. 6.21** | Name mangling to enable type-safe linkage. (Part 1 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
__Z6squarei
__Z6squared
__Z8nothing1ifcRi
__Z8nothing2ciRfRd
main
```

**Fig. 6.21** | Name mangling to enable type-safe linkage. (Part 2 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.16 Function Overloading (cont.)

- For GNU C++, each mangled name (other than main) begins with two underscores (___) followed by the letter `Z`, a number and the function name.
  - The number specifies how many characters are in the function's name.
- The compiler distinguishes the two `square` functions by their parameter lists—one specifies `i` for `int` and the other `d` for `double`.
- The return types of the functions are not specified in the mangled names.
- Overloaded functions can have different return types, but if they do, they must also have different parameter lists.
- Function-name mangling is compiler specific.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Common Programming Error 6.9

*Creating overloaded functions with identical parameter lists and different return types is a compilation error.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Common Programming Error 6.10

*A function with default arguments omitted might be called identically to another overloaded function; this is a compilation error. For example, having a program that contains* both *a function that explicitly takes no arguments and a function of the same name that contains all default arguments results in a compilation error when an attempt is made to use that function name in a call passing no arguments. The compiler cannot determine which version of the function to choose.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.17 Function Templates

- If the program logic and operations are *identical* for each data type, overloading may be performed more compactly and conveniently by using function templates.
- You write a single function template definition.
- Given the argument types provided in calls to this function, C++ automatically generates separate function template specializations to handle each type of call appropriately.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.17 Function Templates (cont.)

- Figure 6.22 defines a `maximum` function that determines the largest of three values.
- All function template definitions begin with the `template` keyword followed by a template parameter list enclosed in angle brackets (< and >).
- Every parameter in the template parameter list is preceded by keyword `typename` or keyword `class`.
- The type parameters are placeholders for fundamental types or user-defined types.
  - Used to specify the types of the function's parameters, to specify the function's return type and to declare variables within the body of the function definition.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.22: maximum.h
2  // Function template maximum header.
3  template <typename T>  // or template<class T>
4  T maximum(T value1, T value2, T value3) {
5     T maximumValue{value1}; // assume value1 is maximum
6
7     // determine whether value2 is greater than maximumValue
8     if (value2 > maximumValue) {
9        maximumValue = value2;
10    }
11
12    // determine whether value3 is greater than maximumValue
13    if (value3 > maximumValue) {
14       maximumValue = value3;
15    }
16
17    return maximumValue;
18 }
```

**Fig. 6.22** | Function template `maximum` header.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.19 Function Templates (cont.)

- Figure 6.23 uses the `maximum` function template to determine the largest of three `int` values, three `double` values and three `char` values, respectively.
- Separate functions are created as a result of the calls—expecting three `int` values, three `double` values and three `char` values, respectively.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.23: fig06_23.cpp
2  // Function template maximum test program.
3  #include <iostream>
4  #include "maximum.h" // include definition of function template maximum
5  using namespace std;
6
7  int main() {
8     // demonstrate maximum with int values
9     cout << "Input three integer values: ";
10    int int1, int2, int3;
11    cin >> int1 >> int2 >> int3;
12
13    // invoke int version of maximum
14    cout << "The maximum integer value is: "
15       << maximum(int1, int2, int3);
16
17    // demonstrate maximum with double values
18    cout << "\n\nInput three double values: ";
19    double double1, double2, double3;
20    cin >> double1 >> double2 >> double3;
21
```

**Fig. 6.23** | Function template `maximum` test program. (Part 1 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
22      // invoke double version of maximum
23      cout << "The maximum double value is: "
24         << maximum(double1, double2, double3);
25
26      // demonstrate maximum with char values
27      cout << "\n\nInput three characters: ";
28      char char1, char2, char3;
29      cin >> char1 >> char2 >> char3;
30
31      // invoke char version of maximum
32      cout << "The maximum character value is: "
33         << maximum(char1, char2, char3) << endl;
34   }
```

```
Input three integer values: 1 2 3
The maximum integer value is: 3

Input three double values: 3.3 2.2 1.1
The maximum double value is: 3.3

Input three characters: A C B
The maximum character value is: C
```

**Fig. 6.23** | Function template `maximum` test program. (Part 2 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.18 Recursion

- A recursive function is a function that calls itself, either directly, or indirectly (through another function).
- Recursive problem-solving approaches have a number of elements in common.
  - A recursive function is called to solve a problem.
  - The function actually knows how to solve only the *simplest case(s)*, or so-called base case(s).
  - If the function is called with a base case, the function simply returns a result.
  - If the function is called with a more complex problem, it typically divides the problem into two conceptual pieces—a piece that the function knows how to do and a piece that it does not know how to do.
  - This new problem looks like the original, so the function calls a copy of itself to work on the smaller problem—this is referred to as a recursive call and is also called the recursion step.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.



## Common Programming Error 6.11

*Omitting the base case or writing the recursion step incorrectly so that it does not converge on the base case causes an infinite recursion error, typically causing a stack overflow. This is analogous to the problem of an infinite loop in an iterative (nonrecursive) solution.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.18 Recursion (cont.)

- The recursion step often includes the key-word `return`, because its result will be combined with the portion of the problem the function knew how to solve to form the result passed back to the original caller, possibly `main`.
- The recursion step executes while the original call to the function is still "open," i.e., it has not yet finished executing.
- The recursion step can result in many more such recursive calls.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.18 Recursion (cont.)

***Factorial***

- The factorial of a nonnegative integer *n, written n! (and pronounced "n factorial"), is the product*
  - $n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 1$
- with 1! equal to 1, and 0! defined to be 1.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.18 Recursion (cont.)

***Iterative Factorial***

- The factorial of an integer, `number`, greater than or equal to 0, can be calculated iteratively (nonrecursively) by using a loop.

***Recursive Factorial***

- A *recursive* definition of the factorial function is arrived at by observing the following algebraic relationship:
  - $n! = n \cdot (n - 1)!$

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.18 Recursion (cont.)

***Evaluating 5!***

- The evaluation of 5! would proceed as shown in Fig. 6.24, which illustrates how the succession of recursive calls proceeds until 1! is evaluated to be 1, terminating the recursion.
- Figure 6.24(b) shows the values returned from each recursive call to its caller until the final value is calculated and returned.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

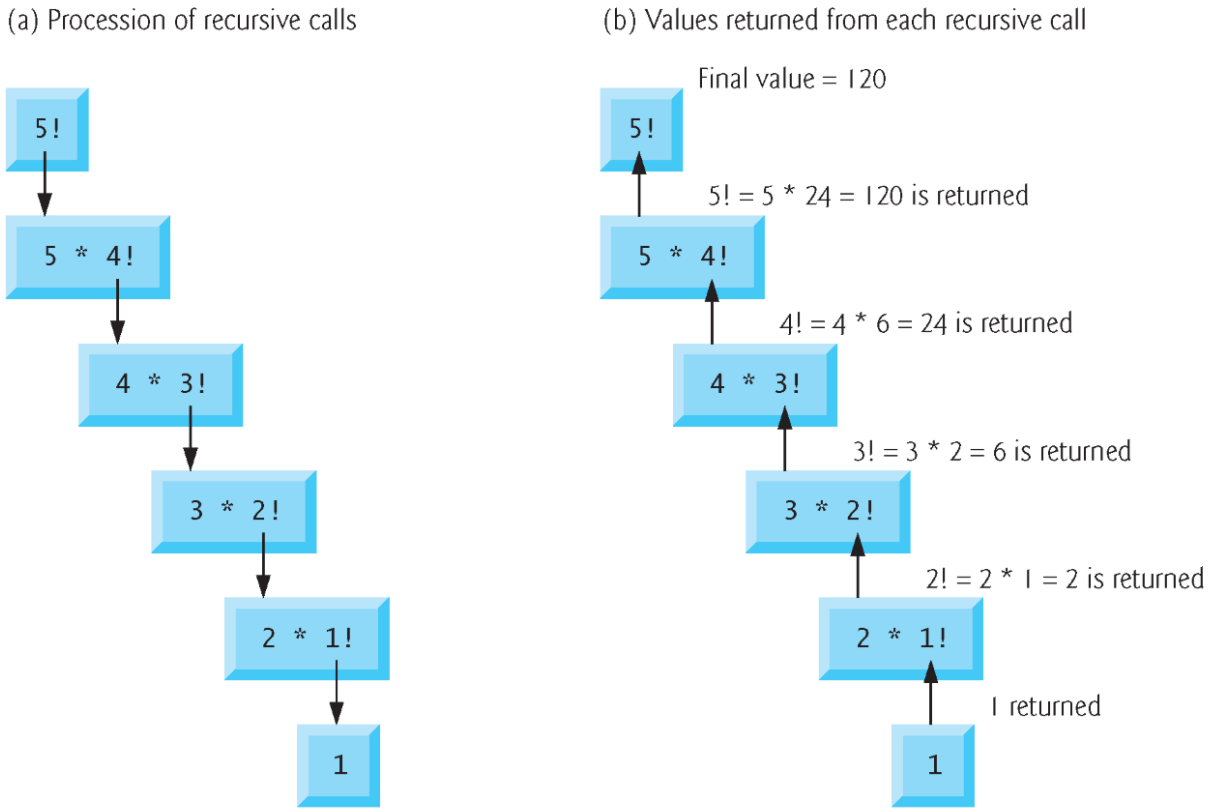



**Fig. 6.24** | Recursive evaluation of 5!.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.18 Recursion (cont.)

***Using a Recursive `factorial` Function to Calculate Factorials***

- Figure 6.25 uses recursion to calculate and print the factorials of the integers 0–10.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.25: fig06_25.cpp
2  // Recursive function factorial.
3  #include <iostream>
4  #include <iomanip>
5  using namespace std;
6
7  unsigned long factorial(unsigned long); // function prototype
8
9  int main() {
10    // calculate the factorials of 0 through 10
11    for (unsigned int counter{0}; counter <= 10; ++counter) {
12       cout << setw(2) << counter << "! = " << factorial(counter)
13          << endl;
14    }
15 }
16
```

**Fig. 6.25** | Recursive function `factorial`. (Part 1 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
17  // recursive definition of function factorial
18  unsigned long factorial(unsigned long number) {
19     if (number <= 1) { // test for base case
20        return 1; // base cases: 0! = 1 and 1! = 1
21     }
22     else { // recursion step
23        return number * factorial(number - 1);
24     }
25  }
```

```
 0! = 1
 1! = 1
 2! = 2
 3! = 6
 4! = 24
 5! = 120
 6! = 720
 7! = 5040
 8! = 40320
 9! = 362880
10! = 3628800
```

**Fig. 6.25** | Recursive function `factorial`. (Part 2 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.18 Recursion (cont.)

***C++11 Type `unsigned long long int`***

C++11's `unsigned long long int` type (which can be abbreviated as `unsigned long long` ) on some systems enables you to store values in at least 8 bytes (64 bits) which can hold numbers as large as 18,446,744,073,709,551,615.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.19 Example Using Recursion: Fibonacci Series

- The Fibonacci series
  - 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
- begins with 0 and 1 and has the property that each subsequent Fibonacci number is the sum of the previous two Fibonacci numbers.
- The series occurs in nature and, in particular, describes a form of spiral.
- The ratio of successive Fibonacci numbers converges on a constant value of 1.618....
- This frequently occurs in nature and has been called the golden ratio or the golden mean.
- Humans tend to find the golden mean aesthetically pleasing.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.19 Example Using Recursion: Fibonacci Series (cont.)

***Recursive Fibonacci Definition***

- The Fibonacci series can be defined recursively as follows:
  - fibonacci(0) = 0
    fibonacci(1) = 1
    fibonacci(*n*) = *fibonacci(n – 1) + fibonacci(n – 2)*
- The program of Fig. 6.26 calculates the *nth Fibonacci number recursively by using function* `fibonacci`.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.26: fig06_26.cpp
2  // Recursive function fibonacci.
3  #include <iostream>
4  using namespace std;
5
6  unsigned long fibonacci(unsigned long); // function prototype
7
8  int main() {
9     // calculate the fibonacci values of 0 through 10
10    for (unsigned int counter{0}; counter <= 10; ++counter)
11       cout << "fibonacci(" << counter << ") = "
12          << fibonacci(counter) << endl;
13
14    // display higher fibonacci values
15    cout << "\nfibonacci(20) = " << fibonacci(20) << endl;
16    cout << "fibonacci(30) = " << fibonacci(30) << endl;
17    cout << "fibonacci(35) = " << fibonacci(35) << endl;
18 }
19
```

**Fig. 6.26** | Recursive function `fibonacci`. (Part 1 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
20  // recursive function fibonacci
21  unsigned long fibonacci(unsigned long number) {
22     if ((0 == number) || (1 == number)) { // base cases
23        return number;
24     }
25     else { // recursion step
26        return fibonacci(number - 1) + fibonacci(number - 2);
27     }
28  }
```

**Fig. 6.26** | Recursive function `fibonacci`. (Part 2 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
fibonacci(0) = 0
fibonacci(1) = 1
fibonacci(2) = 1
fibonacci(3) = 2
fibonacci(4) = 3
fibonacci(5) = 5
fibonacci(6) = 8
fibonacci(7) = 13
fibonacci(8) = 21
fibonacci(9) = 34
fibonacci(10) = 55

fibonacci(20) = 6765
fibonacci(30) = 832040
fibonacci(35) = 9227465
```

**Fig. 6.26** | Recursive function `fibonacci`. (Part 3 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.19 Example Using Recursion: Fibonacci Series (cont.)

- Figure 6.27 shows how function `fibonacci` would evaluate `fibonacci(3)`.
- Figure raises interesting issues about the *order* in which C++ compilers evaluate the operands of operators.
  - *Separate* issue from the order in which operators are applied to their operands, namely, the order dictated by the rules of operator precedence and associativity.
- Most programmers assume that operands are evaluated left to right.
- C++ does not specify the order in which the operands of most operators (including +) are to be evaluated.
- Therefore, you must make no assumption about the order in which these calls execute.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

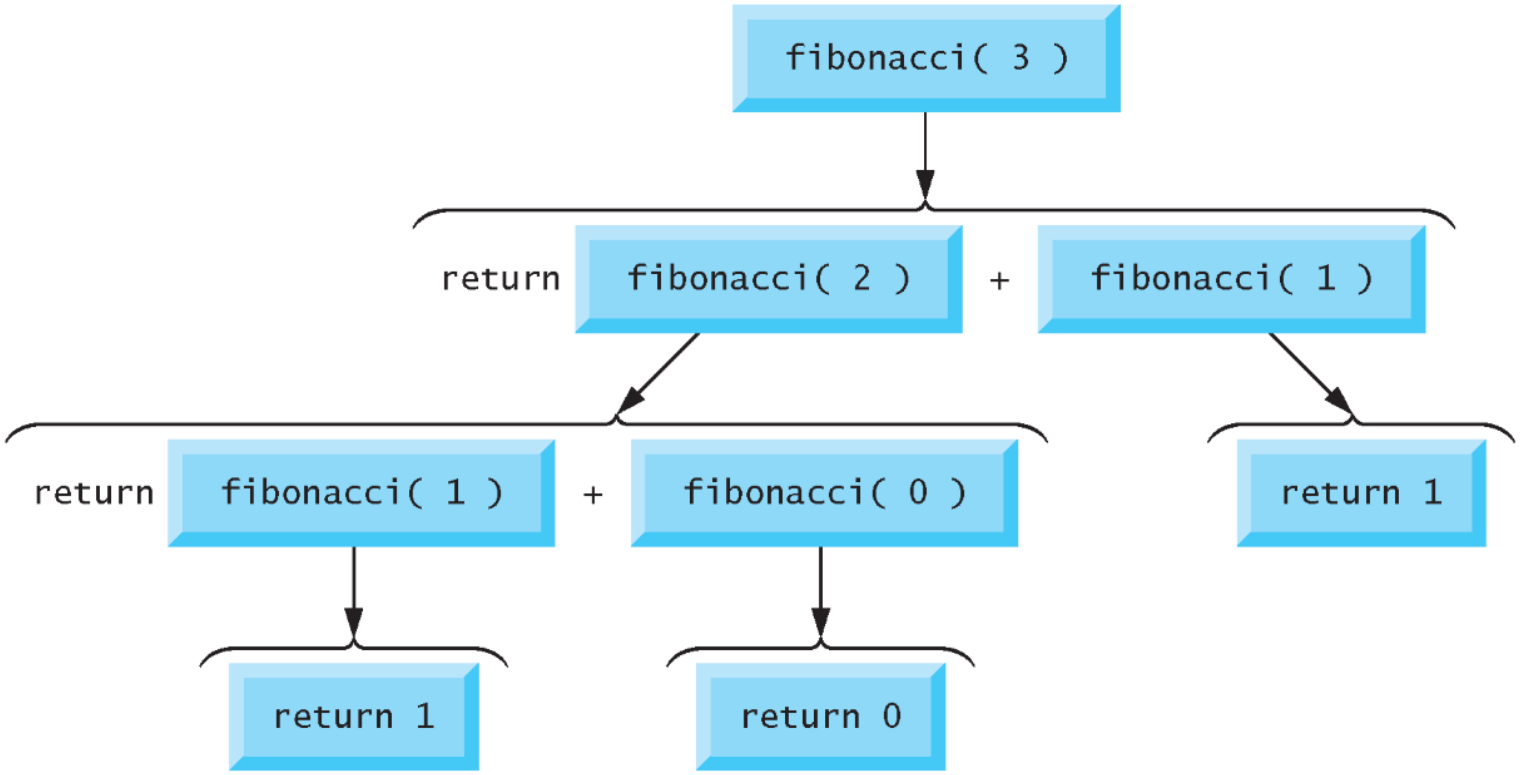



**Fig. 6.27** | Set of recursive calls to function `fibonacci`.

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Portability Tip 6.3

*Programs that depend on the order of evaluation of the operands of operators other than* `&&`, `||`, `?:` *and the comma (*`,`*) operator can function differently with different compilers and can lead to logic errors.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Common Programming Error 6.12

*Writing programs that depend on the order of evaluation of the operands of operators other than &&, ||, ?: and the comma (,) operator can lead to logic errors.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Error-Prevention Tip 6.12

*Do not depend on the order in which operands are evaluated. To ensure that side effects are applied in the correct order, break complex expressions into separate statements.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Common Programming Error 6.13

*Recall that the && and || operators use short-circuit evaluation. Placing an expression with a side effect on the right side of a && or || operator is a logic error if that expression should always be evaluated.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



**Performance Tip 6.5**

*Avoid Fibonacci-style recursive programs that result in an exponential "explosion" of calls.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

# 6.20 Recursion vs. Iteration

- Both iteration and recursion are *based on a control statement*:
  - Iteration uses an *iteration statement*
  - Recursion uses a *selection statement*.
- Both iteration and recursion involve *iteration*:
  - Iteration explicitly uses an *iteration statement*
  - Recursion achieves repetition through *repeated function calls*.
- Iteration and recursion each involve a *termination test*:
  - Iteration terminates when the *loop-continuation condition* fails
  - Recursion terminates when a *base case is recognized*.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

# 6.22 Recursion vs. Iteration (cont.)

- Counter-controlled iteration and recursion both gradually approach termination:
  - Iteration modifies a counter until the counter assumes a value that makes the loop-continuation condition fail
  - Recursion produces *simpler versions of the original problem* until the base case is reached.
- Both iteration and recursion *can occur infinitely*:
  - An *infinite loop* occurs with iteration if the loop-continuation test never becomes false
  - *Infinite recursion* occurs if the recursion step does not reduce the problem during each recursive call in a manner that converges on the base case.
- To illustrate the differences between iteration and recursion, Fig. 6.28 presents an iterative solution to the factorial problem.

©1992–2017 by Pearson Education, Inc.
All Rights Reserved.

```
1  // Fig. 6.28: fig06_28.cpp
2  // Iterative function factorial.
3  #include <iostream>
4  #include <iomanip>
5  using namespace std;
6
7  unsigned long factorial(unsigned int); // function prototype
8
9  int main() {
10    // calculate the factorials of 0 through 10
11    for (unsigned int counter{0}; counter <= 10; ++counter) {
12       cout << setw(2) << counter << "! = " << factorial(counter)
13          << endl;
14    }
15 }
16
```

**Fig. 6.28** | Iterative function `factorial`. (Part 1 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

```
17  // iterative function factorial
18  unsigned long factorial(unsigned int number) {
19     unsigned long result{1};
20
21     // iterative factorial calculation
22     for (unsigned int i{number}; i >= 1; --i) {
23        result *= i;
24     }
25
26     return result;
27  }
```

```
 0! = 1
 1! = 1
 2! = 2
 3! = 6
 4! = 24
 5! = 120
 6! = 720
 7! = 5040
 8! = 40320
 9! = 362880
10! = 3628800
```

**Fig. 6.28** | Iterative function `factorial`. (Part 2 of 2.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Software Engineering Observation 6.11

*Any problem that can be solved recursively can also be solved iteratively (nonrecursively). A recursive approach is normally chosen when the recursive approach more naturally mirrors the problem and results in a program that's easier to understand and debug. Another reason to choose a recursive solution is that an iterative solution may not be apparent when a recursive solution is.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Performance Tip 6.6

*Avoid using recursion in performance situations. Recursive calls take time and consume additional memory.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.



## Common Programming Error 6.14

*Accidentally having a nonrecursive function call itself, either directly or indirectly (through another function), is a logic error.*

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

| Location in text | Recursion examples and exercises |
|---|---|
| *Chapter 6* | |
| Section 6.18, Fig. 6.25 | Factorial function |
| Section 6.19, Fig. 6.26 | Fibonacci function |
| Exercise 6.36 | Recursive exponentiation |
| Exercise 6.38 | Towers of Hanoi |
| Exercise 6.40 | Visualizing recursion |
| Exercise 6.41 | Greatest common divisor |
| Exercise 6.43, Exercise 6.44 | "What does this program do?" exercise |

**Fig. 6.29** | Summary of recursion examples and exercises in the text. (Part 1 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

| Location in text | Recursion examples and exercises |
|---|---|
| *Chapter 7* | |
| Exercise 7.17 | "What does this program do?" exercise |
| Exercise 7.20 | "What does this program do?" exercise |
| Exercise 7.28 | Determine whether a string is a palindrome |
| Exercise 7.29 | Eight Queens |
| Exercise 7.30 | Print an `array` |
| Exercise 7.31 | Print a `string` backward |
| Exercise 7.32 | Minimum value in an `array` |
| Exercise 7.33 | Maze traversal |
| Exercise 7.34 | Generating mazes randomly |

**Fig. 6.29** | Summary of recursion examples and exercises in the text. (Part 2 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.

| Location in text | Recursion examples and exercises |
|---|---|
| *Chapter 19* | |
| Section 19.6, Figs. 19.20–19.22 | Binary tree insert |
| Section 19.6, Figs. 19.20–19.22 | Preorder traversal of a binary tree |
| Section 19.6, Figs. 19.20–19.22 | Inorder traversal of a binary tree |
| Section 19.6, Figs. 19.20–19.22 | Postorder traversal of a binary tree |
| Exercise 19.20 | Print a linked list backward |
| Exercise 19.21 | Search a linked list |
| Exercise 19.22 | Binary tree search |
| Exercise 19.23 | Level order traversal of a binary tree |
| Exercise 19.24 | Printing tree |
| *Chapter 20* | |
| Section 20.3.3, Fig. 20.6 | Mergesort |
| Exercise 20.8 | Linear search |
| Exercise 20.9 | Binary search |
| Exercise 20.10 | Quicksort |

**Fig. 6.29** | Summary of recursion examples and exercises in the text. (Part 3 of 3.)

©1992–2017 by Pearson Education, Inc. All Rights Reserved.